

Heap Sort

By

Prof. Shaik Naseera

Department of CSE

JNTUA College of Engg., Kalikiri

Introduction

- **Heapsort** is a [comparison-based sorting algorithm](#).
- Heapsort can be thought of as an improved [selection sort](#):
- Heap sort uses heap data structure.
- Heap sort divides its elements input into a sorted and an unsorted region, and it iteratively shrinks the unsorted region by extracting the largest element and moving that to the sorted region.

Heap?

- Definitions of **heap**:
 1. A large area of memory from which the programmer can allocate blocks as needed, and de-allocate them (or allow them to be garbage collected) when no longer needed
 2. A balanced, left-justified binary tree in which each node contains the value, is greater than or equal to its children.
- Heap sort uses the second definition

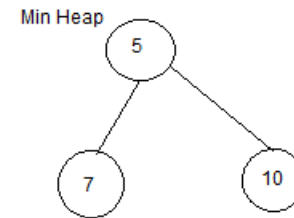
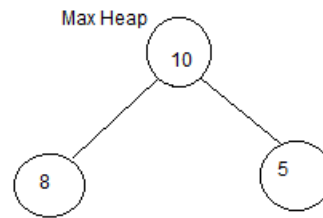
Procedure

Heap Sort Steps:

1. Construction of heap
2. Sorting using reheapifying

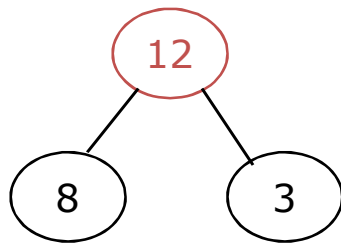
Construction of Heap

- Max Heap
- Min Heap

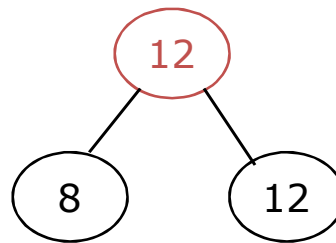


The heap property

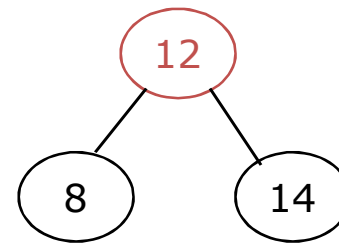
- A node has the **heap property** if the value in the node is as large as or larger than the values of its children



Red node has heap property



Red node has heap property

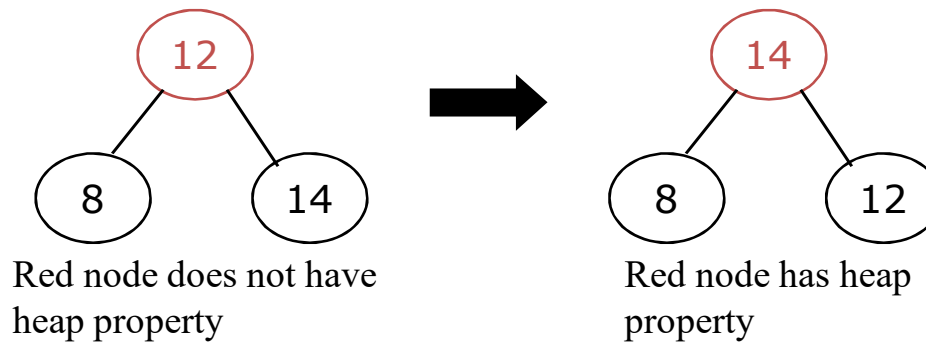


Red node does not have heap property

- All leaf nodes automatically have the heap property
- A binary tree is a **heap** if *all* nodes in it have the heap property

siftUp

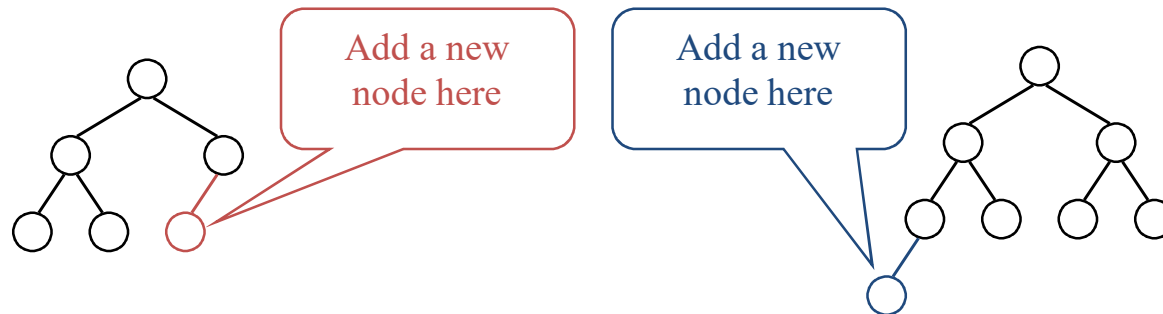
- Given a node that does not have the heap property, you can give it the heap property by exchanging its value with the value of the larger child



- This is sometimes called **sifting up**
- Notice that the child may have *lost* the heap property

Constructing a heap I

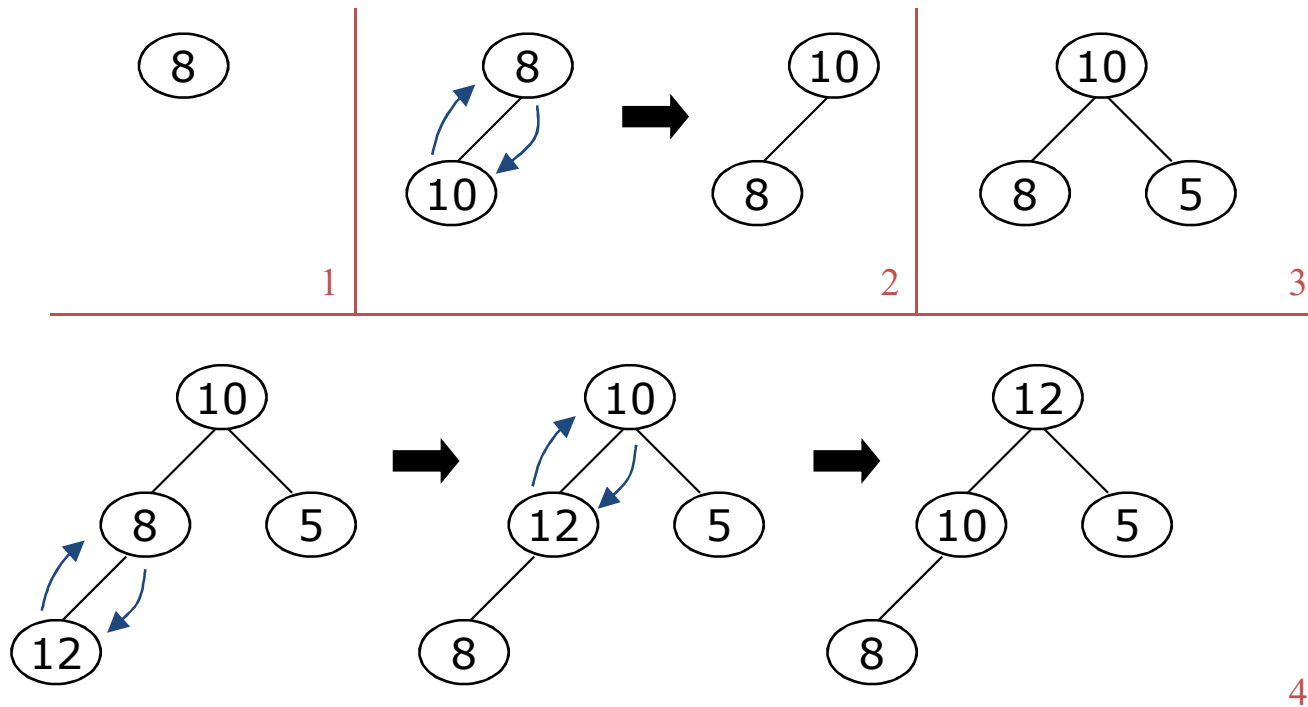
- A tree consisting of a single node is automatically a heap
- We construct a heap by adding nodes one at a time:
 - Add the node just to the right of the rightmost node in the deepest level
 - If the deepest level is full, start a new level
- Examples:



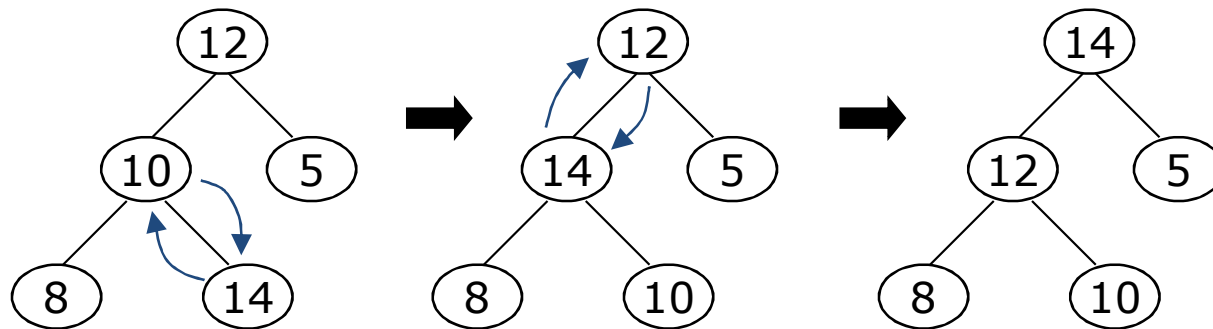
Constructing a heap II

- Each time we add a node, we may destroy the heap property of its parent node
- To fix this, we sift up
- But each time we sift up, the value of the top most node in the shift may increase, and this may destroy the heap property of *its* parent node
- We repeat the shifting up process, moving up in the tree, until either
 - We reach nodes whose values don't need to be swapped (because the parent is *still* larger than both children), or
 - We reach the root

Example: Constructing a heap

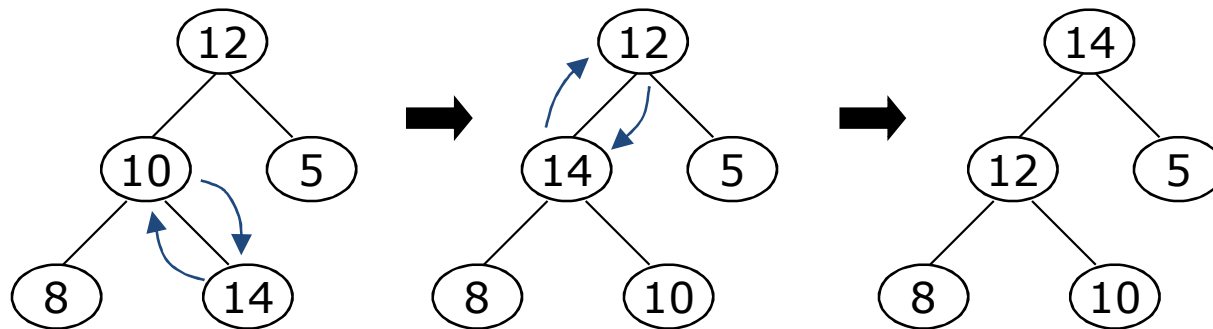


Other children are not affected



- The node containing 8 is not affected because its parent gets larger, not smaller
- The node containing 5 is not affected because its parent gets larger, not smaller
- The node containing 8 is still not affected because, although its parent got smaller, its parent is still greater than it was originally

Other children are not affected



- The node containing 8 is not affected because its parent gets larger, not smaller
- The node containing 5 is not affected because its parent gets larger, not smaller
- The node containing 8 is still not affected because, although its parent got smaller, its parent is still greater than it was originally

Example

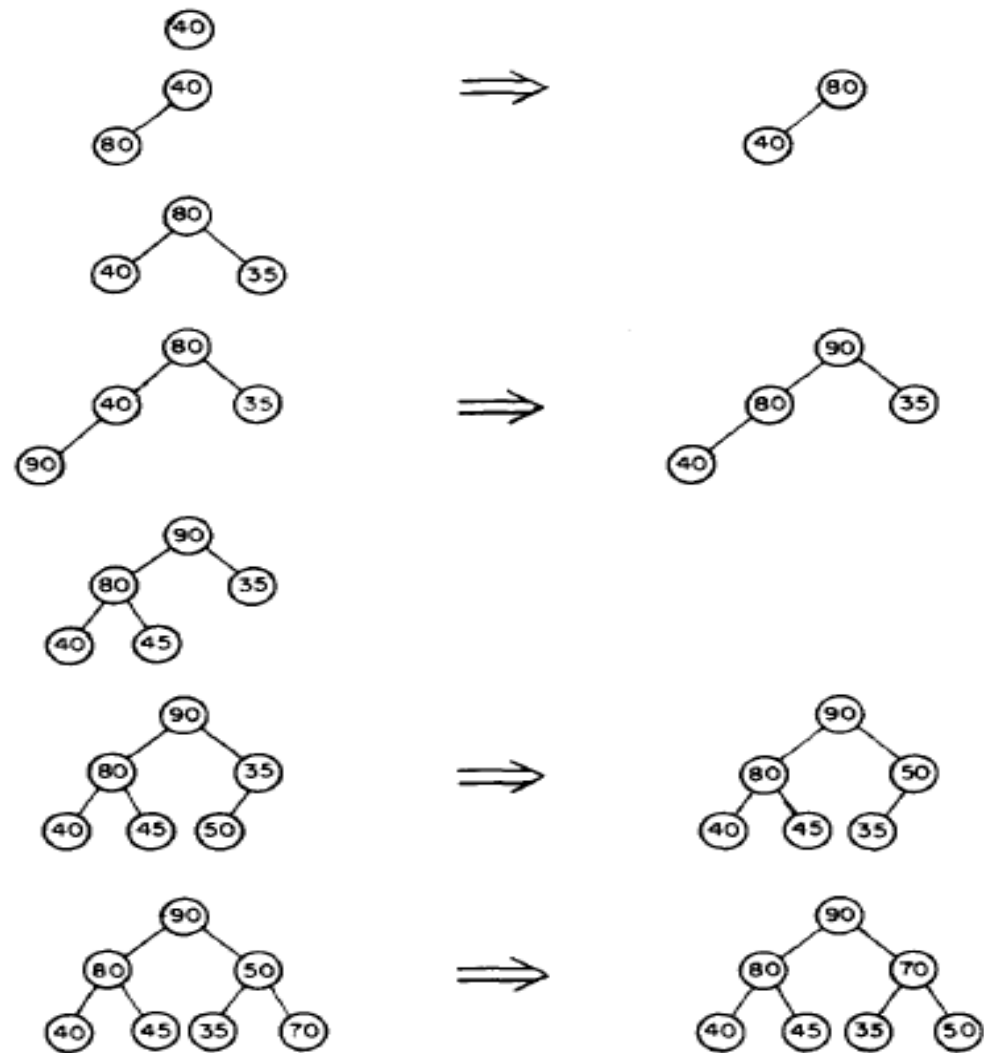
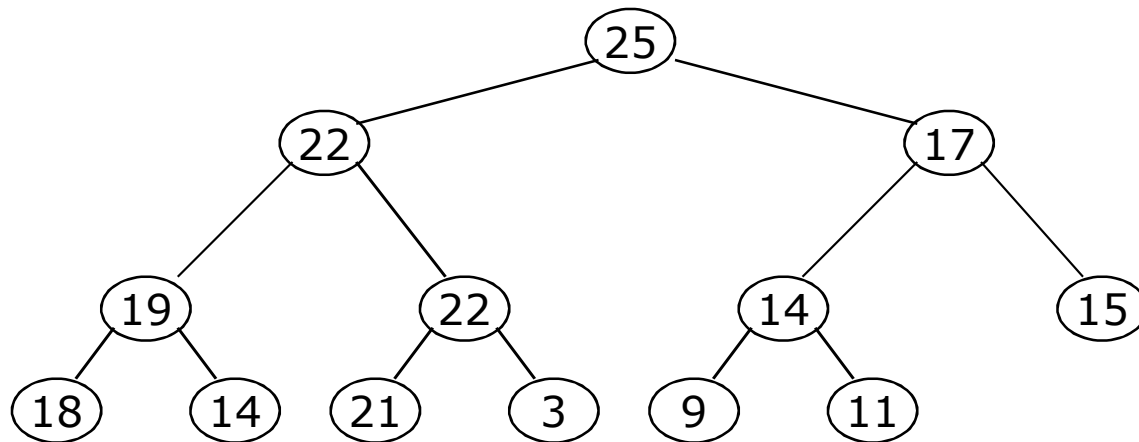


Figure 2.15 Forming a heap from the set (40,80,35,90,45,50,70)

A sample heap

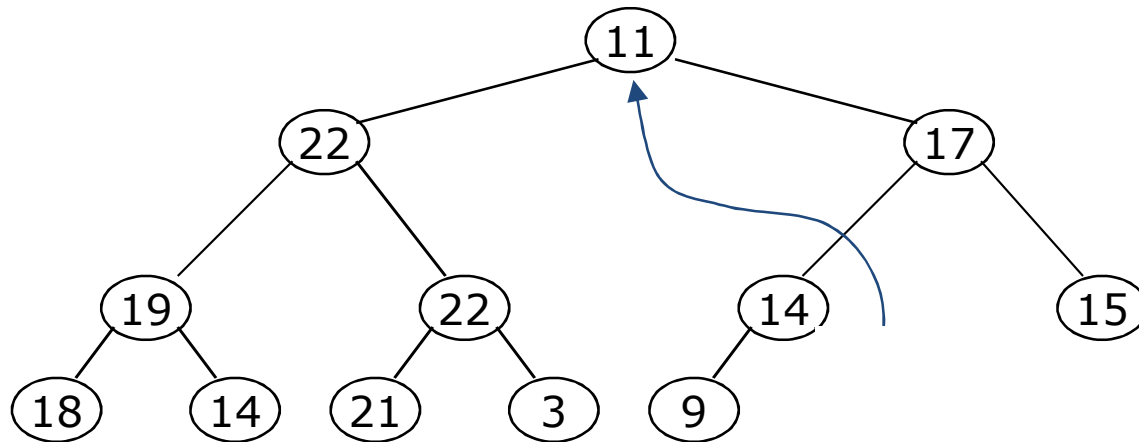
- Here's a sample binary tree after it has been heapified



- Notice that heapified does *not* mean sorted
- Heapifying does *not* change the shape of the binary tree; this binary tree is balanced and left-justified because it started out that way

Removing the root (animated)

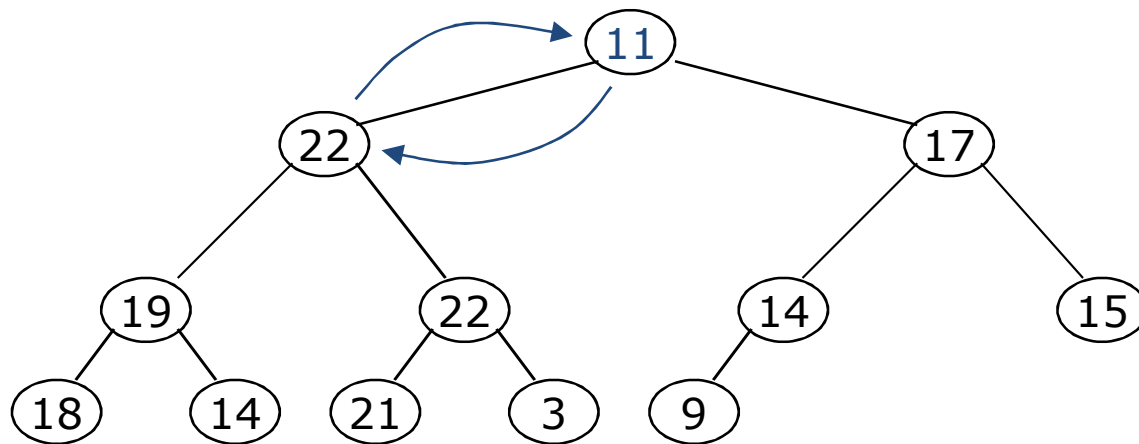
- Notice that the largest number is now in the root
- Suppose we *discard* the root:



- How can we fix the binary tree so it is once again *balanced and left-justified*?
- Solution: remove the rightmost leaf at the deepest level and use it for the new root

The re-Heapify method

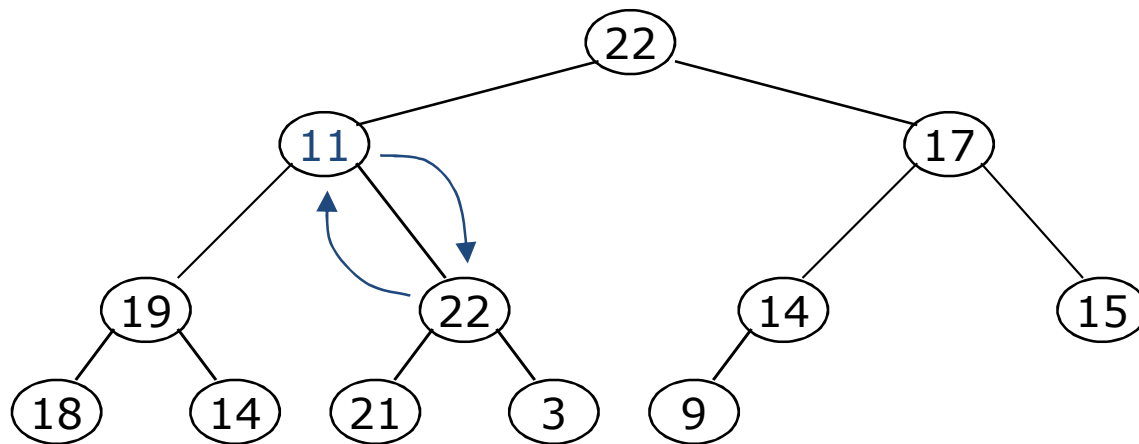
- Our tree is balanced and left-justified, but no longer a heap
- However, *only the root* lacks the heap property



- We can **siftUp()** the root
- After doing this, one and only one of its children may have lost the heap property

The re-Heapify method (cont..)

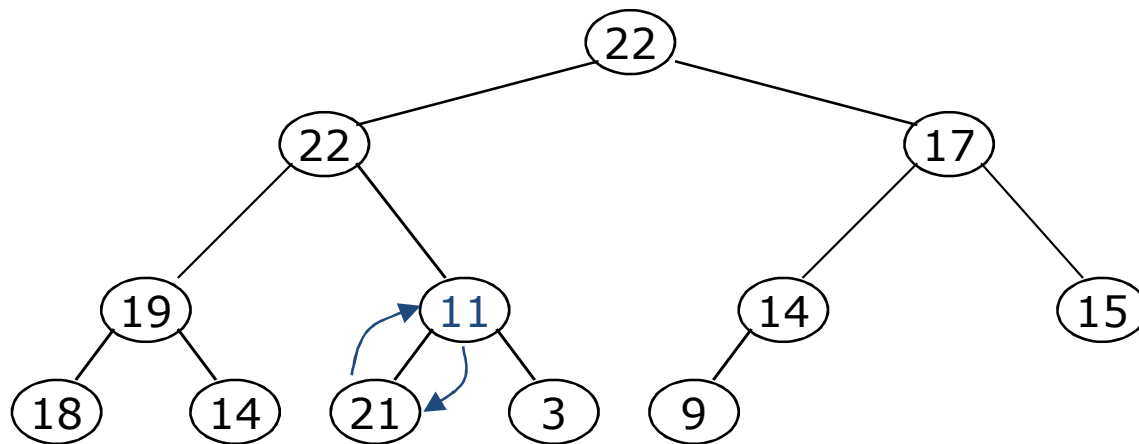
- Now the left child of the root (still the number 11) lacks the heap property



- We can **siftUp()** this node
- After doing this, one and only one of its children may have lost the heap property

The re-Heapify method (cont..)

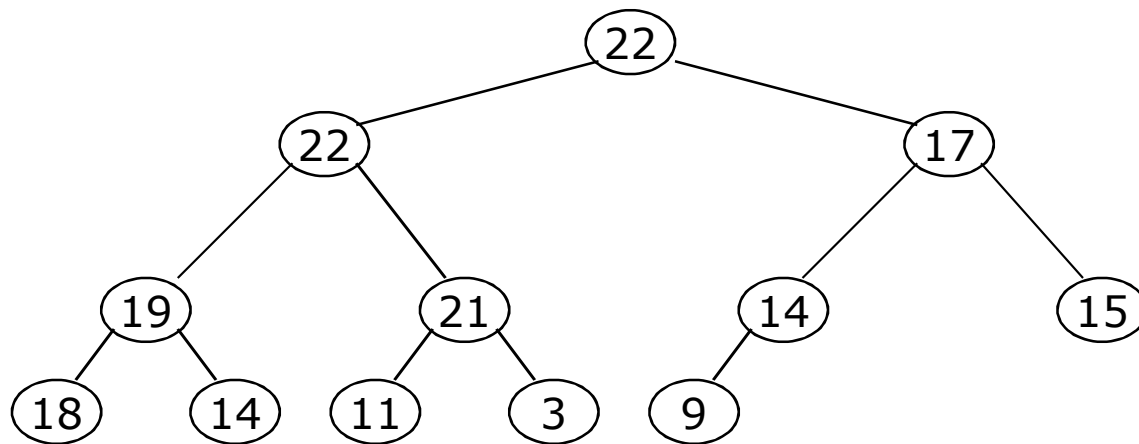
- Now the right child of the left child of the root (still the number 11) lacks the heap property:



- We can **siftUp()** this node
- After doing this, one and only one of its children may have lost the heap property —but it doesn't, because it's a leaf

The re-Heapify method (cont..)

- Our tree is once again a heap, because every node in it has the heap property



- Once again, the largest (or *a* largest) value is in the root
- We can repeat this process until the tree becomes empty
- This produces a sequence of values in order largest to smallest

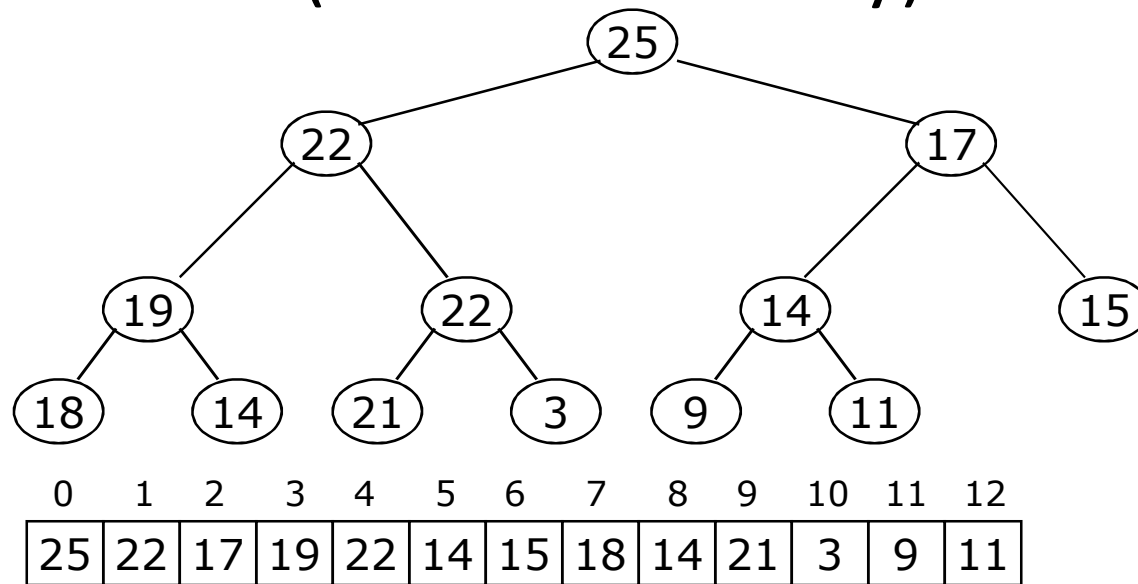
Heap Sort Algorithm

- Contains two steps
- In the first step, **a heap is built** out of the data. The heap is often placed in an array with the layout of a complete binary tree.
- In the second step, **a sorted array is created** by repeatedly removing the largest element from the heap (the root of the heap), and inserting it into the array. The heap is updated after each removal to maintain the heap. Once all objects have been removed from the heap, the result is a sorted array.

Building Heap tree

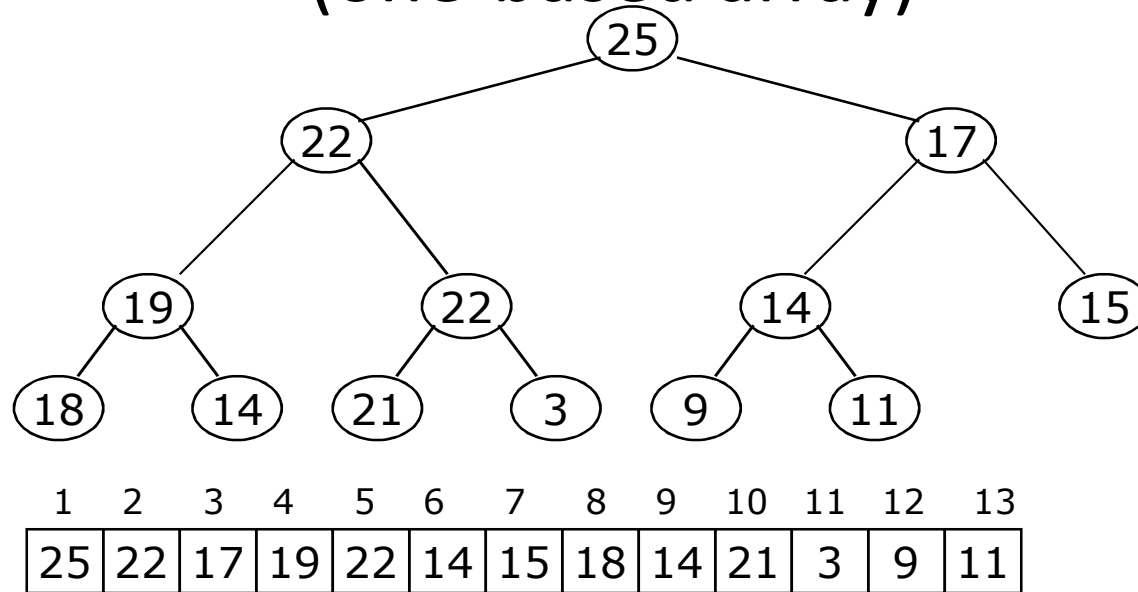
- Pointer Implementation
- Array Implementation
 - For a zero-based array, the root node is stored at index 0;
 - if i is the index of the current node, then
$$i\text{'s LeftChild} = 2*i + 1$$
$$i\text{'s RightChild} = 2*i + 2$$
$$i\text{'s Parent} = \text{floor}((i-1) / 2)$$
 - For a one-based array, the root node is stored at index 1;
 - if i is the index of the current node, then
$$i\text{'s LeftChild} = 2*i$$
$$i\text{'s RightChild} = 2*i + 1$$
$$i\text{'s Parent} = \text{floor}(i / 2)$$

Mapping elements into an array (zero-based array)



- Notice:
 - The left child of index i is at index $2*i+1$
 - The right child of index i is at index $2*i+2$
 - Example: the children of node 3 (19) are 7 (18) and 8 (14)
 - Example: The parent of node 5 (14) is 2 (17) $\text{floor}(i-1/2)$

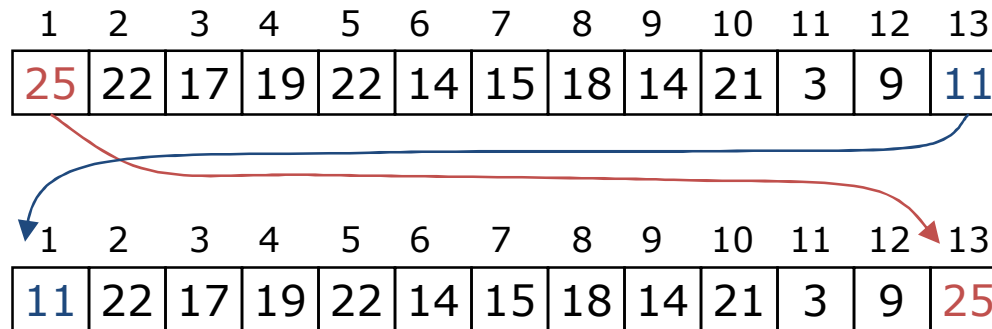
Mapping elements into an array (one-based array)



- Notice:
 - The left child of index i is at index $2*i$
 - The right child of index i is at index $2*i+1$
 - Example: the children of node 4 (19) are 8 (18) and 9 (14)
 - The parent of node 7 (15) is 3 (17) $\text{floor}(i/2)$

Removing and replacing the root

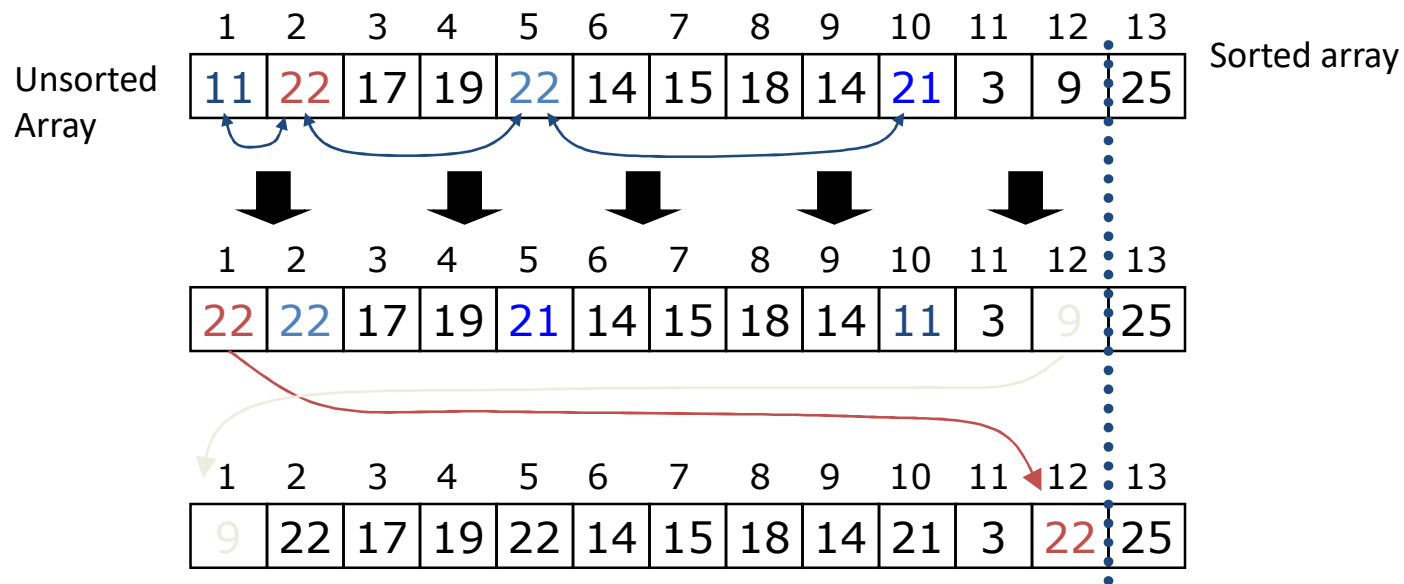
- The “root” is the first element in the array
- The “rightmost node at the deepest level” is the last element
- Swap them...



- ...And pretend that the last element in the array no longer exists—that is, the “last index” is 12 (containing the value 9)

Reheap and repeat

- Reheap the root node (index 1, containing 12)...



- ...And again, remove and replace the root node
- Remember, though, that the “last” array index is changed
- Repeat until the last becomes first, and the array is sorted!

Animation

- [Heapsort-animation.gif](#)

Heap Sort Algorithm

```
procedure HeapSort(B[ ], n)
1 size ← 0;
2 for i ← 1 to n do
3   | MaxHeapInsert (A, size, B[i]);
4 end
5 for i ← n downto 1 do
6   | B[i] ← HeapExtractMax(A, size);
7 end
```

- Time complexity: $O(n \lg n)$

Heap Construction for n elements : $n \lg n$

For sorting n elements : $n \lg n$

Analysis: Construction of heap

- Heapifying the array: we add each of n nodes
 - Each node has to be sifted up, possibly as far as the root
 - Since the binary tree is perfectly balanced, sifting up a single node takes $O(\log n)$ time
 - Since we do this n times, heapifying takes $n * O(\log n)$ time, that is, $O(n \log n)$ time

Analysis: Sorting with Reheapifying

- Here's the rest of the algorithm:

```
while the array isn't empty {  
    remove and replace the root;  
    reheapify the new root node;  
}
```
- We do the while loop n times (actually, $n-1$ times), because we remove one of the n nodes each time
- Removing and replacing the root takes $O(1)$ time and to reheap the root node, it takes $O(\log n)$.
- Removing and reheapifying n nodes take $O(n \log n)$.

Analysis (cont..)

- Here's the algorithm again:

```
heapify the array; //construction of array
while the array isn't empty { //reheapifying
    remove and replace the root;
    reheapify the new root node;
}
```

- We have seen that heapifying (construction of n nodes) takes $O(n \log n)$ time
- The while loop (for removing root node for n nodes) takes $O(n \log n)$ time
- The total time is therefore $O(n \log n) + O(n \log n)$
- This is the same as $O(n \log n)$ time

Complexity

- To insert a single node in an empty heap is : $O(1)$.
- To insert a single element in a n node heap is : $O(\log n)$.
- To insert n elements in a n node heap is : $O(n \log n)$.
- To delete the largest element in a max heap tree : $O(1)$.
- To delete the smallest element in a max heap tree : $O(\log n)$.
- To delete n elements in a max heap tree : $O(n \log n)$.
- To create a heap, time complexity will be : $O(n \log n)$.

Previous Gate Questions

GATE CSE 2015 Set 3

Q. No. 1

Consider the following array of elements.

$\langle 89, 19, 50, 17, 12, 15, 2, 5, 7, 11, 6, 9, 100 \rangle$

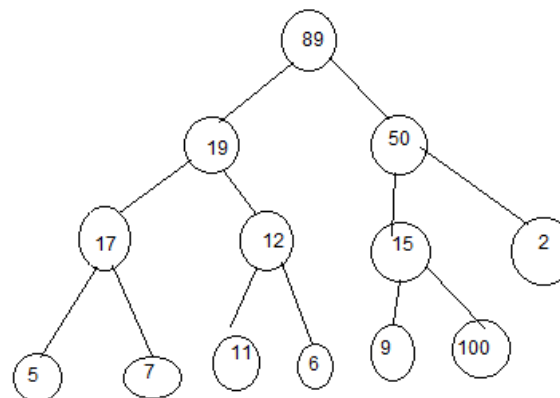
The minimum number of interchanges needed to convert it into a max-heap is

A 4

B 5

C 2

D 3



Q. No. 2

Consider the following statements:

- I. The smallest element in a max-heap is always at a leaf node
- II. The second largest element in a max-heap is always a child of the root node
- III. A max-heap can be constructed from a binary search tree in $\Theta(n)$ time
- IV. A binary search tree can be constructed from a max-heap in $\Theta(n)$ time

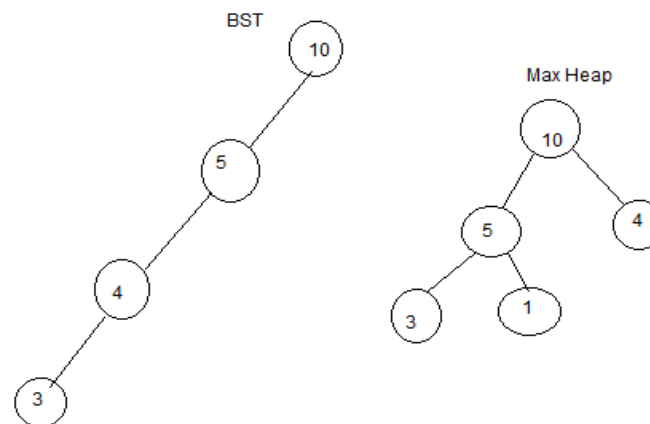
Which of the above statements are TRUE?

A I, II and III

B I, III and IV

C I, II and IV

D II, III and IV



If the form of the binary search tree is left skewed or right skewed, then it requires $O(n^2)$ time

GATE CSE 2014 Set 2

Q. No. 3

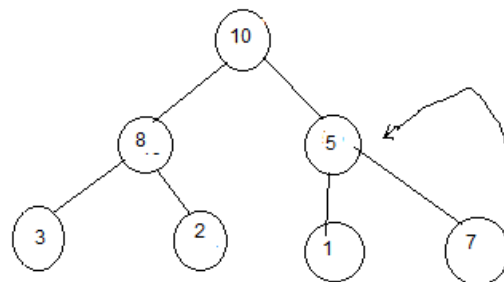
A priority queue is implemented as a Max-Heap. Initially, it has 5 elements. The level-order traversal of the heap is: 10, 8, 5, 3, 2. Two new elements 1 and 7 are inserted into the heap in that order. The level-order traversal of the heap after the insertion of the elements is:

A 10, 8, 7, 3, 2, 1, 5

B 10, 8, 7, 2, 3, 1, 5

C 10, 8, 7, 1, 2, 3, 5

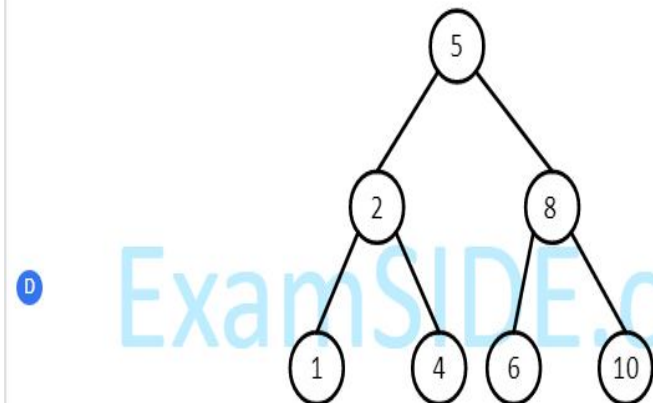
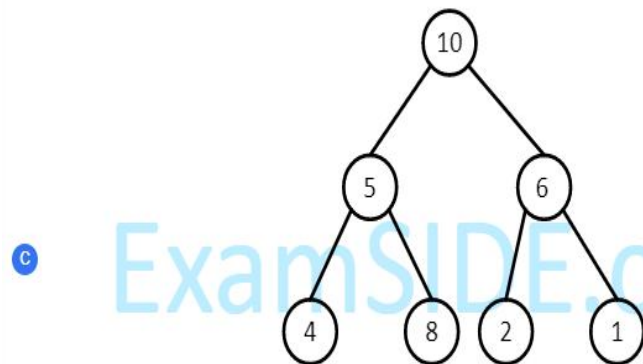
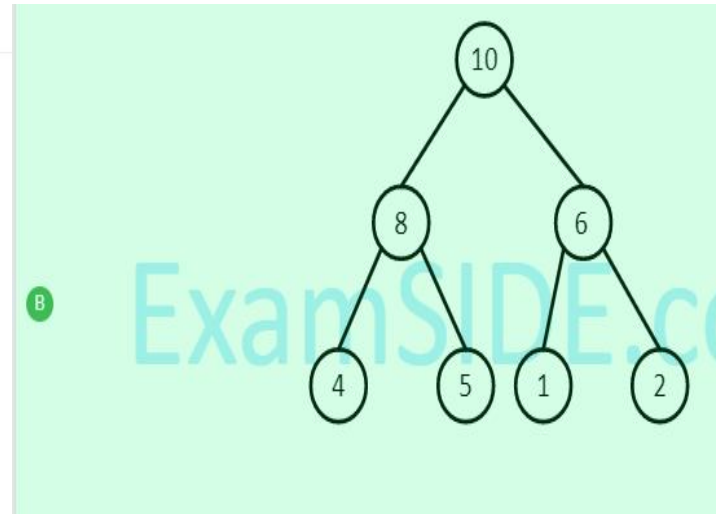
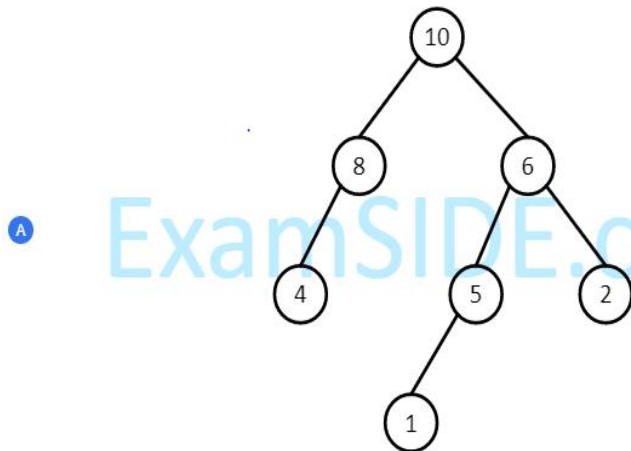
D 10, 8, 7, 5, 3, 2, 1



Q. No. 4

GATE CSE 2011

A max-heap is a heap where the value of each parent is greater than or equal to the value of its children. Which of the following is a max-heap?



GATE CSE 2006

Q. No.5

In a binary max heap containing n numbers, the smallest element can be found in time

A $O(n)$

10

8

6

5

4

1

2

3

B $O(\log n)$

C $O(\log \log n)$

D $O(1)$

From $n/2 + 1$ Location, perform $n/2$ comparisons

Explanation

In max heap, the smallest element should not have any children so it will be present in the leaf node. And to find the smallest element you have to go to the leaf and find minimum among them.

In a heap leaf node starts from $\left\lfloor \frac{n}{2} \right\rfloor + 1$ and end at n . So no of leaf nodes present in the heap is $O\left(\frac{n}{2}\right)$. So total no of comparison needed in order to find the smallest element is $O(n)$ and the time taken for that will be $O(n)$.

Q. No. 6

GATE CSE 2001

Consider any array representation of an n element binary heap where the elements are stored from index 1 to index n of the array. For the element stored at index i of the array ($i \leq n$), the index of the parent is

A $i - 1$

B $\lfloor \frac{i}{2} \rfloor$

C $\lceil \frac{i}{2} \rceil$

D $\frac{i+1}{2}$

GATE CSE 2016 Set 1

Q. No. 7

An operator $delete(i)$ for a binary heap data structure is to be designed to delete the item in the i -th node. Assume that the heap is implemented in an array and i refers to the i -th index of the array. If the heap tree has depth d (number of edges on the path from the root to the farthest leaf), then what is the time complexity to re-fix the heap efficiently after the removal of the element?

A $O(1)$

B $O(d)$ but not $O(1)$

C $O(2^d)$ but not $O(d)$

A maximum of d elements need to be shifted up. Therefore it is $O(d)$

D $O(d2^d)$ but not $O(2^d)$

Q. No. 8

GATE CSE 2015 Set 1

Consider a max heap, represented by the array: 40, 30, 20, 10, 15, 16, 17, 8, 4.

Array Index	1	2	3	4	5	6	7	8	9
Value	40	30	20	10	15	16	17	8	4

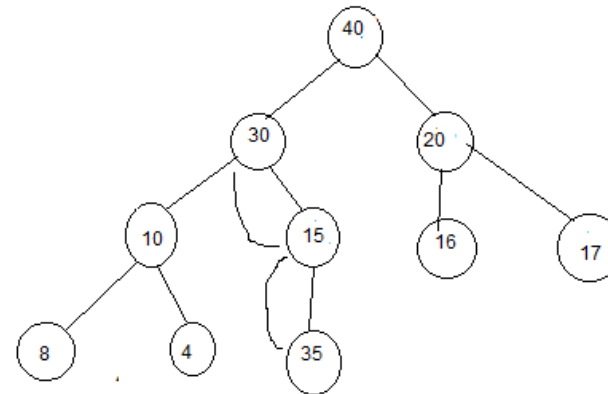
Now consider that a value 35 is inserted into this heap. After insertion, the new heap is

A 40, 30, 20, 10, 15, 16, 17, 8, 4, 35

B 40, 35, 20, 10, 30, 16, 17, 8, 4, 15

C 40, 30, 20, 10, 35, 16, 17, 8, 4, 15

D 40, 35, 20, 10, 15, 16, 17, 8, 4, 30



Q. No. 9

GATE CSE 2009

Consider a binary max-heap implemented using an array. Which one of the following array represents a binary max-heap?

A {25, 12, 16, 13, 10, 8, 14}

B {25, 14, 13, 16, 10, 8, 12}

C {25, 14, 16, 13, 10, 8, 12}

D {25, 14, 12, 13, 10, 8, 16}

Q. No. 10

GATE CSE 2009

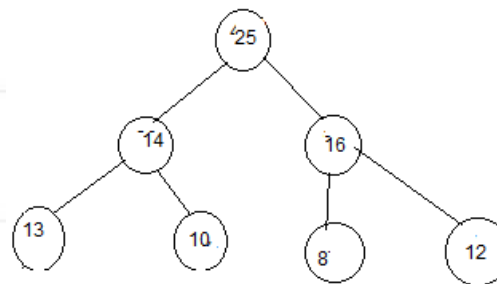
Consider a binary max-heap implemented using an array. What is the content of the array {25, 14, 16, 13, 10, 8, 12} after two delete operations?

A {14, 13, 12, 10, 8}

B {14, 12, 13, 8, 10}

C {14, 13, 8, 12, 10}

D {14, 13, 12, 8, 10}



Delete 25, delete 16

Q. No. 11

GATE CSE 2008

We have a binary heap on n elements and wish to insert n more elements (not necessarily one after another) into this heap. The total time required for this is

A $\Theta(\log n)$

B $\Theta(n)$

To insert n elements in a Binary Heap requires n operations

C $\Theta(n \log n)$

D $\Theta(n^2)$

Q. No. 12

GATE CSE 2005

Suppose there are $\lceil \log n \rceil$ sorted lists of $\left\lfloor \frac{n}{\log n} \right\rfloor$ elements each. The time complexity of producing a sorted list of all these elements is :
(Hint : Use a heap data structure)

A $O(n \log \log n)$

B $\Theta(n \log n)$

C $\Omega(n \log n)$

D $\Omega(n^{3/2})$

Q. No. 13

GATE CSE 1996

The minimum number of interchanges needed to convert the array

89, 19, 40, 17, 12, 10, 2, 5, 7, 11, 6, 9, 70

into a heap with the maximum element at the root is

A 0

B 1

C 2

D 3

Q. No. 14

Consider the following statements:

- I. The smallest element in a max-heap is always at a leaf node
- II. The second largest element in a max-heap is always a child of the root node
- III. A max-heap can be constructed from a binary search tree in $\Theta(n)$ time
- IV. A binary search tree can be constructed from a max-heap in $\Theta(n)$ time

Which of the above statements are TRUE?

☒ A I, II and III

☐ B I, III and IV

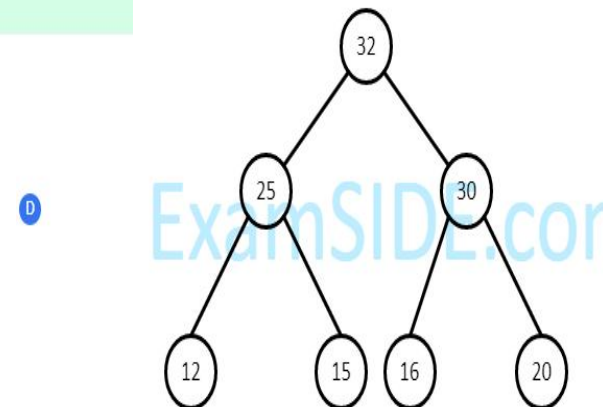
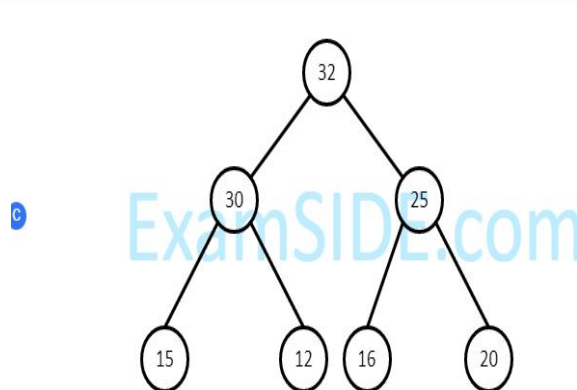
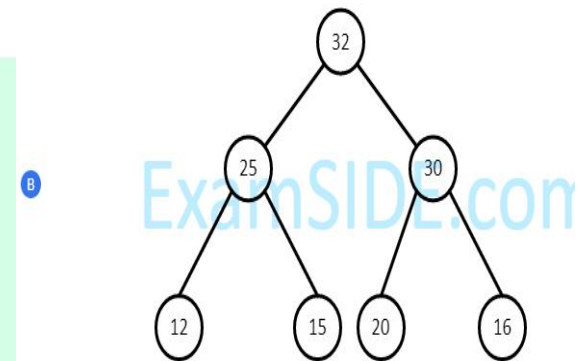
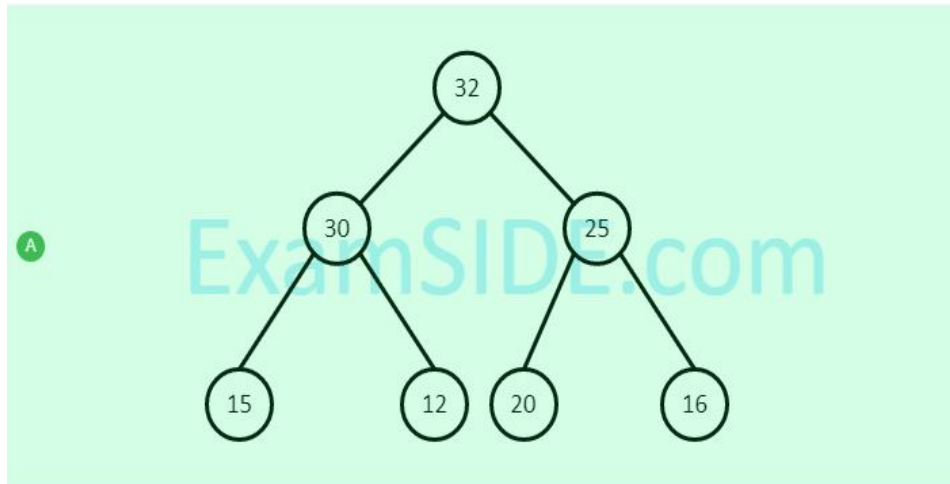
☐ C I, II and IV

☐ D II, III and IV

Q. NO. 15

GATE CSE 2004

The elements 32, 15, 20, 30, 12, 25, 16, are inserted one by one in the given order into a max Heap. The resultant max Heap is



Q. No. 16

GATE CSE 2004

The tightest lower bound on the number of comparisons, in the worst case, for comparison-based sorting is of the order of

A n

B n^2

C $n \log n$

D $n \log^2 n$

Q. No. 17

GATE CSE 2007

Consider the process of inserting an element into a Max Heap, where the Max Heap is represented by an array. Suppose we perform a binary search on the path from the new leaf to the root to find the position for the newly inserted element, the number of comparisons performed is:

- A $\Theta(\log_2 n)$
- B $\Theta(\log_2 \log_2 n)$
- C $\Theta(n)$
- D $\Theta(n \log_2 n)$

Generally $\log n$ comparisons are required on the path of the node.

If Binary search is used, it requires $\log \log n$ comparisons on the path of the node

Thank You